

YPGym Web Application

60-Day Feature Development Plan

Based on the Final Integrated BRD, NoSQL Architecture and Recommended Codebase Structure

Project Type	Full-stack gym membership, attendance, class booking, billing, CRM, personalization and AI support web application
Frontend	React + Vite + TypeScript + Tailwind CSS + shadcn/ui
Backend	Python FastAPI + MongoDB + Redis + JWT + background workers
Architecture Pattern	Monorepo; feature-based frontend; layered backend: API -> Service -> Repository -> Database
Planning Horizon	60 development days
Main Goal	Build every listed feature in a clean order that supports an examiner-ready final year project demo

1. Purpose of This 60-Day Plan

This document converts the YPGym BRD into a day-by-day development plan. It is written from a project management perspective, but the instructions are technical enough for implementation. Each day has one clear focus, the relevant codebase area, implementation instructions and a checkpoint that defines when the day is complete.

The plan assumes the project uses the recommended codebase structure: a monorepo containing a React frontend, a FastAPI backend, MongoDB as the primary NoSQL database, Redis for fast temporary data, and documentation folders for architecture and API notes.

2. Recommended Codebase Structure to Follow

```
ypgym/
├── frontend/
│   ├── src/
│   │   ├── app/
│   │   ├── components/
│   │   ├── features/
│   │   │   ├── auth/
│   │   │   ├── member/
│   │   │   ├── memberships/
│   │   │   ├── attendance/
│   │   │   ├── classes/
│   │   │   ├── billing/
│   │   │   ├── personalization/
│   │   │   ├── chatbot/
│   │   │   └── admin/
│   │   ├── hooks/
│   │   ├── lib/
│   │   └── types/
│   ├── backend/
│   │   ├── app/
│   │   │   ├── api/v1/endpoints/
│   │   │   ├── core/
│   │   │   ├── db/
│   │   │   ├── models/
│   │   │   ├── schemas/
│   │   │   ├── repositories/
│   │   │   ├── services/
│   │   │   ├── workers/
│   │   │   ├── utils/
│   │   │   └── tests/
│   └── docs/
│       ├── architecture/
│       ├── api/
│       └── database/
```

3. Development Rules

- Do not place business logic directly inside API routes. Use API -> Service -> Repository -> Database.
- Frontend pages should stay inside their feature folder. For example, QR code pages belong in features/attendance, not in a random pages folder.
- Every backend feature should have a schema, service and repository when it touches business logic or MongoDB.
- Every sensitive admin action must write to audit_logs.
- Every member-only feature must check JWT authentication and membership status where relevant.

- Build the project in dependency order: account -> membership -> billing -> QR attendance -> classes -> admin -> analytics -> AI chatbot.
- Keep the MVP demo in mind. A simple working feature is better than a complex incomplete feature.

4. 60-Day Milestone Overview

Days	Milestone	Main Deliverables
Days 1-7	Foundation	Repository, Docker, backend scaffold, frontend scaffold, UI shell, API client, MongoDB/Redis setup
Days 8-18	Account, Auth and Membership	Registration, verification, login, forgot password, profile, plans, purchase, invoices, lifecycle
Days 19-25	Admin CRM and Notifications	CRM, member detail, cancellation, revocation, audit, preferences, reminders, broadcast
Days 26-32	QR Attendance and Crowdedness	Rotating QR, check-in, timeout, capacity crowdedness, attendance management, peak heatmap
Days 33-39	Classes and Personal Trainers	Class CRUD, member booking, waitlist, PT management and PT profile browsing
Days 40-49	Personalization, AI and Reporting	Personalization, recommendations, chatbot, guardrails, system settings, analytics, CSV export
Days 50-60	Testing, Performance, Documentation and Finalization	Tests, security, performance, responsive QA, architecture diagrams, demo data, final polish

Day 1: Project setup and monorepo foundation

Codebase area: ypgym/, docs/, frontend/, backend/

Implementation instructions:

- Create the monorepo root folder named ypgym with frontend, backend and docs folders.
- Add README.md explaining the project purpose, tech stack, local setup and folder structure.
- Add .gitignore for Python, Node, environment files, build folders and local database files.
- Create docs/architecture, docs/api and docs/database folders to keep diagrams and technical notes separate from code.

Done when: Repository opens cleanly in VS Code and the folder structure matches the agreed monorepo design.

Day 2: Local development environment with Docker

Codebase area: docker-compose.yml, backend/.env.example, frontend/.env.example

Implementation instructions:

- Create docker-compose.yml with services for MongoDB, Redis, backend and frontend.
- Expose MongoDB and Redis ports only for local development.
- Create .env.example files for frontend and backend with API URL, MongoDB URI, Redis URL, JWT secret, email settings and AI API placeholder.
- Add a short Run Locally section to README.md with docker compose up instructions.

Done when: MongoDB and Redis containers start successfully and environment variables are documented.

Day 3: FastAPI backend scaffold

Codebase area: backend/app/main.py, config.py, dependencies.py, api/v1/router.py

Implementation instructions:

- Create the FastAPI application entry point in app/main.py.
- Create api/v1/router.py and include a health check endpoint such as GET /api/v1/health.
- Create config.py using pydantic-settings to load environment variables.
- Add CORS middleware so the React frontend can call the backend during development.

Done when: Opening /docs in the browser shows FastAPI Swagger and the health endpoint returns success.

Day 4: React frontend scaffold and base routing

Codebase area: frontend/src/app, frontend/src/features, frontend/src/components

Implementation instructions:

- Create the React Vite TypeScript project inside frontend.
- Install Tailwind CSS, shadcn/ui, React Router, TanStack Query, React Hook Form, Zod, Recharts, Sonner and qrcode.react.
- Create App.tsx, router.tsx and providers.tsx.
- Add placeholder routes for public, member and admin areas.

Done when: Frontend starts with npm run dev and displays a working public home page route.

Day 5: Design system, layouts and navigation shell

Codebase area: frontend/src/components/ui, layout, common

Implementation instructions:

- Set up shadcn/ui components for button, card, input, form, table, dialog, dropdown and toast.
- Create MainLayout for public/member pages and AdminLayout for admin pages.
- Create responsive Navbar and Sidebar components.
- Apply a sleek gym-style visual direction using Tailwind: minimal backgrounds, strong typography, clear spacing and mobile-first layout.

Done when: Public, member and admin pages share consistent layout and remain usable on mobile width.

Day 6: Frontend API client and auth state foundation

Codebase area: frontend/src/lib/axios.ts, queryClient.ts, hooks/useAuth.ts

Implementation instructions:

- Create a reusable Axios client pointing to the backend API base URL.
- Configure TanStack Query provider in providers.tsx.
- Create useAuth hook with current user, token storage, logout and role checking helpers.
- Add basic protected route wrappers for member-only and admin-only pages.

Done when: Protected routes redirect unauthenticated users to login and API calls use one central client.

Day 7: MongoDB connection, Redis connection and indexes

Codebase area: backend/app/db/mongodb.py, redis.py, indexes.py

Implementation instructions:

- Create MongoDB connection logic using Motor or Beanie.
- Create Redis client connection logic for temporary QR tokens, occupancy counters and rate limiting.
- Create indexes for users.email, users.phone, user_memberships.userId, attendance_sessions.userId, classes.startTime and audit_logs.createdAt.
- Call index setup during backend startup.

Done when: Backend connects to MongoDB and Redis, and indexes are visible in the database.

Day 8: User registration backend

Codebase area: backend/app/api/v1/endpoints/auth.py, services/auth_service.py, repositories/user_repository.py

Implementation instructions:

- Create User model/schema with name, email, phone, passwordHash, role, emailVerified and timestamps.
- Implement POST /auth/register to validate duplicate email and phone.
- Hash password using bcrypt or passlib before saving.
- Set new users to emailVerified = false and role = member by default.

Done when: A new account can be created through Swagger and duplicate emails/phones are rejected.

Day 9: Email verification backend and frontend page

Codebase area: backend auth_service, email_verifications collection, frontend/features/auth/pages/VerifyEmailPage.tsx

Implementation instructions:

- Generate a time-limited verification token after registration.
- Store the token hash and expiry in email_verifications.
- Create GET or POST /auth/verify-email endpoint to validate the token and mark the user as verified.
- Create a Verify Email landing page that shows success, expired token or invalid token states.

Done when: User registration creates a verification flow and verified users can access the full account.

Day 10: Login, JWT and role-based access control

Codebase area: backend/core/security.py, jwt.py, permissions.py, frontend/features/auth/LoginPage.tsx

Implementation instructions:

- Implement POST /auth/login with email and password validation.
- Block full access for unverified users and return a clear message.
- Generate access token containing user id and role.
- Create role-based dependency helpers for member, admin and PT access.
- Connect the Login page to the backend and store the token securely enough for the project scope.

Done when: Members and admins can log in, and admin endpoints reject member tokens.

Day 11: Forgot password and reset password flow

Codebase area: backend/password_resets, frontend ForgotPasswordPage and ResetPasswordPage

Implementation instructions:

- Create POST /auth/forgot-password that accepts an email and creates a time-limited reset token.
- Create POST /auth/reset-password that validates token and updates password hash.
- Do not reveal whether the email exists in the response to avoid account enumeration.
- Build forgot password and reset password pages with React Hook Form and Zod validation.

Done when: A user can reset the password using a token and then log in with the new password.

Day 12: Member profile settings and secure update validation

Codebase area: frontend/features/member/ProfileSettingsPage.tsx, backend/endpoints/users.py

Implementation instructions:

- Create GET /users/me to return the authenticated user profile.
- Create PATCH /users/me for updating name, phone, email and password.
- Require current password for password change and email re-verification for email change.
- Build Profile Settings page with validation, success toast and error handling.

Done when: Member can update allowed profile fields and sensitive changes are validated.

Day 13: Membership plans data and system configuration seed

Codebase area: backend/models/membership.py, system_configurations, membership_plans

Implementation instructions:

- Create membership_plans collection with 1 month, 3 months, 6 months, 1 year, 2 years and 3 years plans.
- Add fields for durationDays, basePrice, discountPercent and active status.
- Create initial system_configurations values for gym capacity, QR token duration, attendance timeout, cancellation window and waitlist limit.
- Add a seed script to create default plans and configuration.

Done when: Default membership plans and configuration values exist in MongoDB after running the seed script.

Day 14: Membership plan display UI

Codebase area: frontend/features/memberships/pages/MembershipPlansPage.tsx, components/MembershipCard.tsx

Implementation instructions:

- Create GET /membership-plans endpoint.
- Build MembershipCard component showing duration, price, discount and benefits.
- Display plans in a responsive grid for desktop and mobile.
- Add a Select Plan button that leads authenticated users to purchase and guests to login/register.

Done when: Users can view all active membership plans clearly on desktop and mobile.

Day 15: Membership purchase and mock payment flow

Codebase area: backend/services/membership_service.py, billing_service.py, frontend Buy/RenewMembership page

Implementation instructions:

- Create POST /memberships/purchase that accepts planId and mock payment confirmation.
- Calculate price and discount from the membership plan record, not from frontend input.
- If user has active membership, extend from current expiry date; otherwise start from today.
- Create payment record and user_membership record in one service flow.

Done when: A member can buy or renew a plan and the membership expiry updates correctly.

Day 16: Invoice generation and invoice storage

Codebase area: backend/services/invoice_service.py, workers/invoice_worker.py, invoices collection

Implementation instructions:

- Use ReportLab or WeasyPrint to generate a PDF invoice after successful membership purchase.
- Include invoice id, member name, plan, amount, discount, transaction date, start date and expiry date.
- Store invoice metadata and PDF path/reference in invoices collection.
- Optionally run invoice generation synchronously for MVP or through a background worker if ready.

Done when: A purchase produces a downloadable invoice record linked to the member and payment.

Day 17: Payment and billing history

Codebase area: frontend/features/billing, backend/endpoints/billing.py

Implementation instructions:

- Create GET /billing/me/payments and GET /billing/me/invoices for members.
- Create admin billing endpoints with filters by member, date and status.
- Build PaymentHistoryPage and InvoiceDetailsPage for members.
- Add basic AdminBillingPage table for admin review.

Done when: Members can view their own invoices and admins can inspect billing records.

Day 18: Membership lifecycle: active, expiring, expired and frozen

Codebase area: backend/services/membership_service.py, user_memberships collection

Implementation instructions:

- Implement membership status calculation based on start date, expiry date, freeze period and revocation/cancellation fields.
- Add helper function get_current_membership_status(userId).
- Implement freeze/hold request fields and service logic for Frozen status.

- Document freeze rules in system configuration for examiner clarity.

Done when: Membership status is calculated consistently and Frozen is handled as a real state.

Day 19: Admin CRM member list

Codebase area: frontend/features/admin/pages/MembersCRMPage.tsx, backend/admin endpoints

Implementation instructions:

- Create GET /admin/members with search, status filter, role filter and pagination.
- Use TanStack Table for sorting, filtering and pagination on the frontend.
- Show name, phone, email, membership status, plan type, expiry date and actions.
- Protect the endpoint with admin-only permission.

Done when: Admin can search and filter members from the CRM page.

Day 20: Admin member detail view

Codebase area: frontend/admin components, backend/admin/member detail endpoint

Implementation instructions:

- Create GET /admin/members/{userId} returning profile, membership, attendance summary, bookings and billing summary.
- Build a member detail drawer or page from the CRM table.
- Add tabs for Profile, Membership, Billing, Attendance, Bookings and Audit.
- Keep editing actions separated from viewing actions to reduce accidental admin changes.

Done when: Admin can inspect a member end-to-end from one place.

Day 21: Cancellation request and approval workflow

Codebase area: backend/cancellation_requests, frontend CancellationRequest page and Admin Cancellation Requests page

Implementation instructions:

- Create POST /memberships/cancellation-request for members to submit reason and request type.
- Create admin endpoint to approve or reject cancellation.
- Require admin to choose refund, account credit or forfeit outcome before approval.
- Update membership status to Cancelled only after approval and notify the member.

Done when: Cancellation is traceable, policy-based and not just a direct delete/update.

Day 22: Membership revocation and audit trail

Codebase area: backend/audit_service.py, audit_logs collection, Admin member actions

Implementation instructions:

- Create admin action to revoke membership with required reason field.
- Update membership status to Revoked and block QR/class access.
- Write audit log with admin user id, affected member id, action, reason and timestamp.
- Show revocation action only to admins.

Done when: Revocation is possible, controlled and logged for auditability.

Day 23: Notification preferences

Codebase area: backend/notification_preferences, frontend NotificationPreferencesPage.tsx

Implementation instructions:

- Create notification preference model with emailEnabled, inAppEnabled and supported event types.
- Create GET/PATCH /notifications/preferences/me.
- Build a member page for enabling or disabling supported notification channels.
- Make notification service check preferences before sending non-critical messages.

Done when: Member can manage notification preferences and preferences are stored.

Day 24: Expiry reminders and notification service

Codebase area: backend/services/notification_service.py, workers/expiry_reminder_worker.py

Implementation instructions:

- Create notification collection records for in-app messages.
- Implement email reminder template for membership expiring soon.
- Create scheduled job that finds memberships expiring in 7 days, 3 days and 1 day.
- Avoid duplicate reminders by storing reminderSent flags or notification history.

Done when: Expiring members receive reminders without duplicate spam.

Day 25: Admin broadcast announcements

Codebase area: frontend/AdminBroadcastPage.tsx, backend/broadcast_announcements

Implementation instructions:

- Create admin endpoint to create broadcast announcements with title, message, target audience and expiry date.
- Store announcement records in MongoDB.
- Display active announcements on member dashboard or notification area.
- Write audit log for broadcast creation.

Done when: Admin can send gym-wide messages such as closures or class schedule changes.

Day 26: Rotating QR token backend

Codebase area: backend/services/qr_service.py, Redis, qr_tokens optional collection

Implementation instructions:

- Create an endpoint GET /attendance/qr-token/me for active members.
- Generate a signed short-lived token containing user id, issued time and expiry time.
- Store token id or hash in Redis with TTL if using server-side validation.
- Reject token generation for expired, frozen, cancelled or revoked memberships.

Done when: Backend can issue secure time-limited QR tokens only for eligible members.

Day 27: Member QR code page

Codebase area: frontend/features/attendance/pages/MyQRCodePage.tsx, components/QRCodeDisplay.tsx

Implementation instructions:

- Use qrcode.react to render the current QR token as a QR image.
- Refresh the token automatically before expiry based on configured token duration.

- Show clear status messages when membership is inactive, expired, frozen or revoked.
- Prevent the page from showing a permanent static QR code.

Done when: Member sees a rotating QR code that updates automatically and reflects membership status.

Day 28: Scanner/check-in endpoint

Codebase area: backend/endpoints/attendance.py, services/attendance_service.py

Implementation instructions:

- Create POST /attendance/check-in for scanner/admin use.
- Validate QR token signature, Redis token status, user identity and active membership.
- Create an attendance_sessions document with checkInTime, status active and autoExpiresAt.
- Return success, user name and crowdedness update to the scanner UI or API response.

Done when: A valid QR token creates one active attendance session.

Day 29: Duplicate prevention, check-out and timeout worker

Codebase area: backend/workers/attendance_timeout_worker.py, attendance_service.py

Implementation instructions:

- Prevent repeated check-in if the user already has an active session.
- Create POST /attendance/check-out for optional exit scanning.
- Create scheduled worker to expire sessions where autoExpiresAt is in the past.
- Set session status to timed_out or completed so active occupancy decreases correctly.

Done when: Occupancy does not only increase; sessions can end by check-out or timeout.

Day 30: Capacity-based crowdedness calculation

Codebase area: backend/services/crowdedness_service.py, frontend/CrowdednessIndicator.tsx

Implementation instructions:

- Calculate active sessions where status is active and autoExpiresAt is greater than current time.
- Read gym capacity from system_configurations.
- Convert occupancy to percentage and map to Low, Moderate, Busy or Very Crowded.
- Display crowdedness on member dashboard and admin dashboard.

Done when: Crowdedness uses real active sessions and capacity percentage, not hardcoded absolute numbers.

Day 31: Admin attendance management

Codebase area: frontend/AdminAttendancePage.tsx, backend/admin attendance endpoints

Implementation instructions:

- Create admin endpoint to view active sessions and historical attendance logs.
- Add filters by date, member and status.
- Allow admin to manually close a stuck session with a required reason.
- Write audit log for manual session closure.

Done when: Admin can manage attendance sessions and fix abnormal cases safely.

Day 32: Peak-hours heatmap

Codebase area: frontend/components/charts/PeakHoursHeatmap.tsx, backend analytics endpoint

Implementation instructions:

- Aggregate attendance sessions by day of week and hour of day.
- Create GET /admin/analytics/peak-hours returning heatmap-ready data.
- Render a day-by-hour chart using Recharts or a simple grid heatmap component.
- Add this chart to Reports and Analytics or Admin Dashboard.

Done when: Admin can visually identify peak hours using attendance data already collected.

Day 33: Class CRUD backend

Codebase area: backend/models/gym_class.py, class_repository.py, class_service.py, endpoints/classes.py

Implementation instructions:

- Create classes collection model with title, description, startTime, endTime, capacity, status and assignedTrainerId.
- Create admin endpoints for creating, updating, cancelling and listing classes.
- Validate capacity, date/time and trainer existence.
- Protect write actions with admin-only access.

Done when: Admins can manage class records through API.

Day 34: Admin class management UI

Codebase area: frontend/features/admin/pages/AdminClassesPage.tsx

Implementation instructions:

- Build a table of classes with filters by date, status and trainer.
- Add create/edit class dialog using React Hook Form and Zod.
- Add cancel class action with confirmation dialog.
- Show remaining capacity and waitlist count for each class.

Done when: Admin can create, edit and cancel classes from the dashboard.

Day 35: Member class listing with PT card

Codebase area: frontend/features/classes/pages/ClassesPage.tsx, components/ClassCard.tsx, PTProfileCard.tsx

Implementation instructions:

- Create public/member endpoint to list upcoming available classes.
- Include trainer profile summary, capacity, remaining slots and class status.
- Build a mobile-friendly class card with book button and PT profile preview.
- Show disabled states for inactive membership or full classes.

Done when: Members can browse classes and see assigned trainer information.

Day 36: Class booking and cancellation window

Codebase area: backend/services/class_service.py, frontend/MyBookingsPage.tsx

Implementation instructions:

- Create POST /classes/{classId}/book for active members.
- Prevent booking if membership is inactive, class is full, class is cancelled or user already booked.
- Create DELETE or POST /classes/{classId}/cancel-booking respecting configured cancellation window.
- Build My Bookings page showing upcoming bookings and cancel action when allowed.

Done when: Members can book classes and cancel only within allowed policy.

Day 37: Class waitlist and automatic notification

Codebase area: backend/services/waitlist_service.py, class_waitlists collection

Implementation instructions:

- When class is full, allow member to join waitlist instead of booking.
- Store waitlist order by joinedAt timestamp.
- When a booked member cancels, notify or auto-enrol the first waitlisted member according to policy.
- Add waitlist status to member My Bookings page.

Done when: Full classes have a clear waitlist process and users are notified when a slot opens.

Day 38: PT management for admin

Codebase area: backend/personal_trainers or pt_profiles, frontend/AdminPTPage.tsx

Implementation instructions:

- Create PT profile fields: name, specialty, short bio, availability summary and active status.
- Create admin endpoints to create, update and deactivate PT profiles.
- Build AdminPTPage with table and profile form.
- Allow PT assignment from class management using active PT profiles.

Done when: Admins can manage trainer profiles used in class browsing.

Day 39: Member PT profile browsing

Codebase area: frontend/features/classes/pages/PTProfilesPage.tsx, backend/trainers public/member endpoint

Implementation instructions:

- Create endpoint to list active PT profile cards.
- Show trainer name, specialty, assigned upcoming classes and availability summary.
- Link PT cards from class details and the PT Profiles page.
- Keep this as a view-only MVP feature unless a full PT dashboard is later approved.

Done when: Members can browse trainer information without requiring a complex PT role implementation.

Day 40: Personalization profile form

Codebase area: backend/personalization_profiles, frontend/PersonalizationFormPage.tsx

Implementation instructions:

- Create personalization profile schema with goals, preferred training style, preferred intensity, habits, preferred class times, PT interest and 1-2 year development targets.
- Restrict submission to active members only.
- Build form with multi-select fields, text areas and validation.
- Store updated profile with timestamp for recommendation logic.

Done when: Active members can submit personalization data for future class recommendations.

Day 41: Class recommendation engine

Codebase area: backend/services/recommendation_service.py, class_recommendations collection

Implementation instructions:

- Create rule-based recommendation logic first rather than complex AI.
- Match personalization goals and preferred intensity with class tags, PT specialty and class availability.
- Use attendance history as a secondary signal when available.
- Return recommended classes with explanation text such as recommended because it matches your strength goal.

Done when: System can recommend classes in a transparent and explainable way.

Day 42: Recommended classes UI and admin personalization insights

Codebase area: frontend/RecommendedClassesPage.tsx, Admin Personalization Insights page

Implementation instructions:

- Build Recommended Classes page using recommendation endpoint.
- Show reason labels for each recommendation.
- Create admin endpoint for aggregated counts of goals, preferred class types and popular time preferences.
- Do not expose unnecessary sensitive user-level fitness details in admin insights.

Done when: Members receive recommendations and admins see safe aggregated planning insights.

Day 43: AI chatbot context builder

Codebase area: backend/services/chatbot_service.py, chatbot_context_logs

Implementation instructions:

- Create backend function build_chatbot_context(userId) that fetches current membership status, expiry date, bookings, crowdedness, invoice pointers and allowed actions.
- Inject this context into the chatbot request before calling the AI provider.
- Do not allow the frontend to send trusted membership facts directly to the model.
- Log only safe metadata about chatbot requests.

Done when: Chatbot answers are grounded in authenticated backend context.

Day 44: Chatbot guardrails and out-of-scope handling

Codebase area: backend/services/guardrail_service.py, tests/test_chatbot_guardrails.py

Implementation instructions:

- Define allowed topics: membership, crowdedness, classes, PT availability, navigation and billing.
- Define restricted topics: exercise advice, diet advice, medical advice and unrelated general questions.
- Before or after AI response, apply a topic check and return fixed guardrail responses for restricted topics.
- Write tests for common out-of-scope prompts.

Done when: Chatbot refuses or redirects restricted questions consistently.

Day 45: Chatbot frontend interface

Codebase area: frontend/features/chatbot/pages/ChatbotPage.tsx, components/ChatWindow.tsx

Implementation instructions:

- Build chat interface with message list, input box, loading state and error handling.
- Send messages to backend chatbot endpoint with authentication.
- Show suggested prompt buttons such as When does my membership expire and Is the gym crowded now.
- Keep UI simple and mobile-friendly.

Done when: Members can ask approved system-related questions through a clean chat UI.

Day 46: Chatbot logs and admin monitoring

Codebase area: backend/chatbot_logs, frontend/admin chatbot monitoring optional section

Implementation instructions:

- Store chatbot query metadata such as userId, topic category, allowed/restricted result, timestamp and error state.
- Create admin view or analytics card showing total chatbot usage and restricted query count.
- Do not store sensitive full conversations unless needed for the project and clearly documented.
- Use logs to demonstrate guardrail behavior during viva.

Done when: Admin can demonstrate chatbot usage and guardrail monitoring safely.

Day 47: System configuration management

Codebase area: frontend/SystemSettingsPage.tsx, backend/system_configurations

Implementation instructions:

- Create admin page to update gym capacity, QR token duration, attendance timeout, cancellation window and waitlist limit.
- Validate configuration values to prevent unsafe settings such as zero capacity or negative timeout.
- Cache frequently used settings in Redis if needed.
- Write audit log whenever an admin changes configuration.

Done when: Admin can configure operational rules without changing code.

Day 48: Operational analytics snapshots

Codebase area: backend/analytics_snapshots, workers/analytics_worker.py, frontend OperationalAnalytics page

Implementation instructions:

- Create scheduled or on-demand aggregation for membership trends, class popularity, attendance patterns and revenue summary.
- Store snapshots in analytics_snapshots to avoid recalculating heavy dashboards repeatedly.
- Render analytics cards and charts in admin dashboard.
- Use Recharts for line, bar and heatmap-style visualizations.

Done when: Admin dashboard has clear operational analytics backed by stored calculations.

Day 49: CSV export for CRM and billing

Codebase area: backend/utils/csv_export.py, frontend/ExportCSVButton.tsx

Implementation instructions:

- Create admin endpoints to export filtered member list and billing table as CSV.
- Reuse the same filters from CRM and Billing pages so exported data matches the screen.
- Add Export CSV buttons to Member CRM and Admin Billing pages.
- Write audit log for exports if treating export as a sensitive action.

Done when: Admin can export member and billing records for reporting.

Day 50: Backend tests for auth and membership

Codebase area: backend/app/tests/test_auth.py, test_membership.py

Implementation instructions:

- Write tests for registration, duplicate email, email verification, login, forgot password and role protection.
- Write tests for membership purchase, expiry calculation, renewal and frozen status.
- Use test database or mocked repositories to keep tests repeatable.
- Run tests through pytest and document command in backend README.

Done when: Core account and membership logic has automated test coverage.

Day 51: Backend tests for attendance and classes

Codebase area: backend/app/tests/test_attendance.py, test_classes.py

Implementation instructions:

- Write tests for QR token validation, duplicate check-in prevention, timeout logic and crowdedness percentage.
- Write tests for class booking, cancellation window and waitlist promotion.
- Add edge cases such as revoked member, expired member and full class.
- Fix any service logic discovered by failing tests.

Done when: Attendance and class operations are protected against obvious logic bugs.

Day 52: Tests for chatbot guardrails and admin actions

Codebase area: backend/app/tests/test_chatbot_guardrails.py, admin tests

Implementation instructions:

- Write tests for allowed chatbot topics and restricted topics.
- Test that exercise, diet and medical advice prompts return the fixed redirect response.
- Test cancellation approval, revocation and system configuration changes create audit logs.
- Add these tests to the regular pytest command.

Done when: High-risk features have evidence of testing for examiner review.

Day 53: Performance tuning, indexes and caching pass

Codebase area: backend/db/indexes.py, Redis services, analytics services

Implementation instructions:

- Review all frequent queries and ensure matching MongoDB indexes exist.
- Cache current crowdedness and configuration values in Redis where suitable.
- Add pagination to admin lists and avoid returning large collections at once.
- Create a short docs/database/performance-notes.md explaining 1000+ user scalability approach.

Done when: System avoids obvious performance problems and the scalability story is documented.

Day 54: Security hardening and validation pass

Codebase area: backend/core, frontend forms, API endpoints

Implementation instructions:

- Add rate limiting for login, forgot password and chatbot endpoints using Redis.
- Confirm all sensitive endpoints require JWT and correct roles.
- Review Zod and Pydantic schemas for required validation.
- Check CORS, environment secrets and error messages so secrets are not exposed.

Done when: Security basics are implemented and common abuse cases are reduced.

Day 55: Responsive UI and PWA readiness

Codebase area: frontend layouts, pages, tailwind config

Implementation instructions:

- Test all public, member and admin pages on mobile, tablet and desktop widths.
- Fix overflow, table scrolling, button spacing and unreadable layouts.
- Add PWA-ready metadata if desired, such as app name and mobile viewport settings.
- Prioritize QR page, dashboard, class booking and chatbot for mobile usability.

Done when: Main user flows are usable on a phone, which is essential for QR check-in.

Day 56: Architecture diagrams and technical documentation

Codebase area: docs/architecture/high-level-architecture.png, data-flow-diagram.png

Implementation instructions:

- Create high-level architecture diagram showing React client, FastAPI API layer, service layer, MongoDB, Redis and external integrations.
- Create data flow diagram for membership purchase and QR attendance.
- Create short docs explaining API -> Service -> Repository -> Database backend pattern.
- Keep diagrams simple enough for lecturer review and viva explanation.

Done when: Architecture is visually documented and matches the implemented codebase.

Day 57: API documentation and developer README

Codebase area: docs/api/api-endpoints.md, backend README, frontend README

Implementation instructions:

- Export or summarize key API endpoints from FastAPI Swagger.
- Document how to run frontend, backend, MongoDB and Redis locally.
- Document test commands, seed commands and demo account credentials.
- Add troubleshooting notes for common setup problems.

Done when: Another person can run and understand the project from the documentation.

Day 58: Demo data and examiner scenario setup

Codebase area: backend/scripts/seed_demo_data.py, docs/demo-script.md

Implementation instructions:

- Create seed data for admin account, member accounts, membership plans, active membership, expired membership, classes, PT profiles and attendance logs.
- Add realistic class bookings and waitlist examples.
- Prepare demo script showing member registration, membership purchase, QR attendance, crowdedness, class booking, admin CRM, analytics and chatbot guardrail.
- Test the demo from a clean database.

Done when: A reliable demo scenario is ready and does not depend on random manual setup.

Day 59: Integration bug fixing and end-to-end polish

Codebase area: whole codebase

Implementation instructions:

- Run through every major user flow from the demo script and record bugs.
- Fix broken navigation, API errors, validation problems and inconsistent statuses.
- Check that toast messages, loading states and empty states are clear.
- Clean unused files and confirm folder naming remains consistent.

Done when: End-to-end demo flows work without major interruptions.

Day 60: Final deployment package and viva checklist

Codebase area: docs, README, final build, screenshots

Implementation instructions:

- Build frontend and confirm backend starts with production-like environment variables.
- Prepare final screenshots of main pages and admin dashboards.
- Create viva checklist explaining why React, FastAPI, MongoDB, Redis, QR rotation, guardrails and layered architecture were chosen.
- Tag a final Git commit and create a backup zip or release version.

Done when: Project is ready for submission, demo and technical explanation.

5. Final Submission Checklist

- ☐ Frontend and backend run locally using documented commands.
- ☐ MongoDB and Redis services start successfully.
- ☐ Seed script creates demo admin, members, plans, classes, PT profiles and attendance data.
- ☐ Member can register, verify email, log in, buy membership, receive invoice and view QR code.
- ☐ QR attendance supports active session creation and timeout or check-out so crowdedness is accurate.
- ☐ Admin can manage members, memberships, classes, PTs, billing, cancellation requests and audit logs.
- ☐ Class booking supports capacity, cancellation window and waitlist.
- ☐ Personalization profile can recommend classes using rule-based logic.
- ☐ Chatbot uses authenticated backend context and rejects out-of-scope exercise, diet and medical advice.
- ☐ CSV export, heatmap, broadcast announcements and operational analytics are demo-ready.
- ☐ Architecture diagrams are included in docs/architecture.
- ☐ Tests cover authentication, membership, attendance, classes, admin audit actions and chatbot guardrails.
- ☐ Final README includes setup, API, demo accounts and viva explanation points.